

Code-Layout, Readability and Reusability

Date	1 July 2026
Team ID	xxxxxx
Project Name	Emotion Detection & Learning Support Engine
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes/No/Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Used standard Python indentation (4 spaces) throughout the project, following PEP8 conventions.
2	Proper File Structure	Files and folders are logically organized	Yes	Project follows a modular structure with separate modules for preprocessing, the BERT model, the BiLSTM model, Gemini AI integration, and the Streamlit UI.
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Variables such as <code>learning_problem</code> , <code>primary_emotion</code> , <code>mixed_emotions</code> , and <code>confidence_score</code> clearly indicate their purpose.
4	Function / Method Names	Functions are descriptively named	Yes	Functions such as <code>predict_emotion_bert()</code> , <code>predict_emotion_bilstm()</code> , <code>generate_guidance()</code> , and <code>generate_study_plan()</code> use meaningful names.
5	Code Comments	Inline and block comments explain logic	Yes	Major sections such as model loading, preprocessing, and Gemini API calls are documented using comments.
6	Modular Design	Code is split into reusable functions/modules	Yes	Backend logic is divided into reusable modules for preprocessing, BERT prediction, BiLSTM prediction, Gemini guidance generation, and history/analytics management.
7	No Redundant Code	Duplicate or unused code is removed	Yes	Common preprocessing and utility functions are shared across the BERT and BiLSTM prediction modules, reducing duplication.
8	Error Handling	Exceptions and errors are handled gracefully	Yes	Try-except blocks handle model loading failures, Gemini API errors, and file read/write operations, with fallback responses when Gemini AI is unavailable.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	Text Preprocessing Module	Python (Regex, NLTK)	Used before both BERT and BiLSTM predictions	High
2	BERT Emotion Prediction Service	Python (Hugging Face Transformers, TensorFlow)	Primary/secondary emotion detection module	High
3	BiLSTM Emotion Prediction Service	Python (TensorFlow, Keras)	Mixed emotion detection module	High
4	Gemini AI Guidance Generator	Python (Google Generative AI SDK)	Learning guidance and study plan generation	High
5	History & Analytics Logger	Python (Pandas, CSV)	Stores interaction history, powers the analytics dashboard	Medium

Overall Code Quality Assessment:

S.No	Quality Aspect	Assessment	Comments
1	Readability	Excellent	Clear naming conventions and consistent formatting throughout the codebase.
2	Modularity	Excellent	Well-separated modules for preprocessing, prediction, guidance generation, and storage.
3	Reusability	Good	Core prediction and preprocessing modules can be reused in other education-focused NLP applications.
4	Maintainability	Good	Modular structure and documented functions make the code easy to maintain and extend.
5	Documentation	Good	Inline comments and docstrings cover the major functions and modules.